Proyecto final de Lenguajes formales

Analizadores LL(1) y SLR(1) y definiciones dirigidas por la sintaxis

Miguel Muñoz Juan José Sierra

Noviembre de 2026

1. Resumen de la entrega

Este informe documenta nuestra entrega del proyecto final. Implementamos el cálculo de FIRST y FOLLOW, un analizador LL(1) con tabla y pila, un analizador SLR(1) y el evaluador de una definición dirigida por la sintaxis (SDD) que usa el grafo de dependencias. Todo el código está en Python.

Dos programas de consola siguen los formatos de entrada y salida del enunciado: `cli/main.py` para los Ítems 1 y 2, y `cli/sdd_cli.py` para el Ítem 3. Un servidor Flask ofrece las mismas funciones en páginas web. No usamos generadores de parsers ni librerías que calculen conjuntos FIRST y FOLLOW, autómatas LR o tablas de análisis. Las instrucciones para instalar y ejecutar todo están en el `README.md`.

2. Secciones del libro y estructuras de datos

Seguimos a Aho, Lam, Sethi y Ullman, *Compilers: Principles, Techniques, and Tools*, segunda edición. La tabla indica qué sección seguimos en cada parte, dónde está el código y qué estructura de datos usa.

Parte	Libro	Código	Estructura de datos
FIRST y FOLLOW	4.4.2	`cfg_grammar.py`	Un conjunto de símbolos para cada símbolo y cada no terminal. Se repite el cálculo hasta que ningún conjunto crece.
Condición LL(1)	4.4.3	`check_ll1_conditions` en `cfg_ll1.py`	Lista de violaciones, una por cada par de producciones del mismo no terminal.
Tabla predictiva	algoritmo 4.31	clase `LL1`	Diccionario $(A, a) \to$ lista de producciones. Una celda con más de una producción es un conflicto.
Analizador predictivo	4.4.4, algoritmo 4.34	`LL1.parse`	Pila de símbolos.
Ítems LR(0), CLOSURE y GOTO	4.6.2	`cfg_slr1.py`	Un ítem es un par (producción, posición del punto). Un estado es un conjunto inmutable de ítems. Las transiciones están en un diccionario (estado, símbolo)→estado. La gramática se aumenta con $S' \to S$.

Parte	Libro	Código	Estructura de datos
ACTION y GOTO	4.6.4	`SLR1._build_tables`	Diccionario (estado, terminal)→ lista de acciones, y lista de conflictos.
Algoritmo de análisis LR	4.6.3	`SLR1.parse`	Pila de pares (símbolo, estado).
SDD y árbol anotado	5.1.1, 5.1.2	`cfg_sdd.py`, `cfg_tree.py`	El SDD son datos: gramática, tabla de atributos y lista de reglas por producción. Cada nodo del árbol guarda su símbolo, su producción, sus hijos y sus atributos.
Grafo de dependencias y orden de evaluación	5.2.1, 5.2.2	`build_graph`, `topological_order`	Lista de aristas (atributo usado → atributo definido). El orden sale del algoritmo de Kahn. Un ciclo se informa y detiene la evaluación.
Clasificación S y L-atribuida	5.2.3, 5.2.4	`SDD.classify`	Recorre las reglas y revisa de qué atributos depende cada atributo heredado.

Los números de sección son los que cita el enunciado para cada parte (secciones 4.4.2 a 4.4.4, 4.5, 4.6.2 a 4.6.4, 5.1 y 5.2).

3. Ítems 1 y 2: FIRST, FOLLOW y los dos analizadores

3.1. La gramática

Elegimos una gramática propia que no aparece en el enunciado. Genera listas de identificadores separados por comas, con grupos entre corchetes. La letra `i` representa un identificador. Pertenecen al lenguaje, por ejemplo, `i`, `i,i` y `[i,i],i`.

$$\begin{array}{lcl} S & \to & L \\ L & \to & \texttt{i}R \;\mid\; \texttt{[}L\texttt{]}R \\ R & \to & \texttt{,i}R \;\mid\; \varepsilon \end{array}$$

En el formato de entrada del enunciado, la gramática se escribe así:

```
3
S -> L
L -> iR [L]R
R -> ,iR e
```

3.2. FIRST y FOLLOW

La tabla sale del programa con la opción `-sets`. R puede derivar la cadena vacía, y por eso ε está en FIRST(R). FOLLOW(R) es igual a FOLLOW(L), porque R está al final de las dos producciones de L. FOLLOW(L) contiene `]`, que sigue a L en $L \to$ `[`L`]`R, y `$`, que viene de $S \to L$.

3.3. Condición LL(1)

Revisamos a mano los pares de producciones del mismo no terminal, como indica la sección 4.4.3.

Cuadro 2: FIRST y FOLLOW (sección 4.4.2)

No terminal	FIRST	FOLLOW
L	{[, i}	{], $}
R	{„, ε}	{], $}
S	{[, i}	{$}

- Para L: FIRST(`i`R) = {`i`} y FIRST(`[`L`]`R) = {`[`}. Los conjuntos no comparten símbolos, y ninguna de las dos alternativas deriva ε.
- Para R: solo la segunda alternativa deriva ε. FIRST(`,i`R) = {`,`} no comparte símbolos con FOLLOW(R) = {`]`, `$`}.
- S tiene una sola producción.

La gramática cumple la condición, así que es LL(1). El programa también encuentra que es SLR(1). Es el caso 1 de la sección 2.2.2 del enunciado, y por eso pregunta qué analizador usar.

3.4. Tabla LL(1)

Ninguna celda tiene más de una producción. La producción $R \to \varepsilon$ ocupa las columnas `]` y `$`, que son los elementos de FOLLOW(R).

Cuadro 3: Tabla de análisis predictivo M de LL(1) (algoritmo 4.31)

No terminal	,	[	]	i	$
L		L → [L]R		L → iR	
R	R → ,iR		R → ε		R → ε
S		S → L		S → L	

3.5. Autómata LR(0) y tabla SLR(1)

Aumentamos la gramática con la producción $S' \to S$, que lleva el número 0. La colección canónica tiene 12 conjuntos de ítems.

Cuadro 4: Colección canónica de conjuntos de ítems LR(0) y transiciones GOTO (sección 4.6.2)

Estado	Ítems	Transiciones (símbolo: estado)
I_0	S′ → ·S S → ·L L → ·iR L → ·[L]R	L: 1, S: 2, [: 3, i: 4
I_1	S → L·	—
I_2	S′ → S·	—
I_3	L → ·iR L → ·[L]R L → [·L]R	L: 5, [: 3, i: 4
I_4	L → i·R R → ·,iR R → ·	,: 6, R: 7
I_5	L → [L·]R	]: 8

Estado	Ítems	Transiciones
I_6	R → ,·iR	i: 9
I_7	L → iR·	—
I_8	L → [L]·R R → ·,iR R → ·	,: 6, R: 10
I_9	R → ·,iR R → ,i·R R → ·	,: 6, R: 11
I_{10}	L → [L]R·	—
I_{11}	R → ,iR·	—

Cuadro 5: Producciones de la gramática aumentada (rn reduce por la producción n)

n	Producción
0	S′ → S
1	S → L
2	L → iR
3	L → [L]R
4	R → ,iR
5	R → ε

La tabla ACTION/GOTO se construye con la colección anterior y con los conjuntos FOLLOW. Ninguna celda tiene más de una acción, y por eso la gramática es SLR(1).

3.6. Una cadena paso a paso

Analizamos la cadena `[i,i],i` con los dos analizadores. Los dos la aceptan en 15 pasos.

Cuadro 7: Análisis LL(1) de `[i,i],i` (sección 4.4.4)

Paso	Pila (la cima queda a la izquierda)	Entrada	Acción
1	`S $`	`[i,i],i$`	emitir S → L
2	`L $`	`[i,i],i$`	emitir L → [L]R
3	`[ L ] R $`	`[i,i],i$`	emparejar [
4	`L ] R $`	`i,i],i$`	emitir L → iR
5	`i R ] R $`	`i,i],i$`	emparejar i
6	`R ] R $`	`,i],i$`	emitir R → ,iR
7	`, i R ] R $`	`,i],i$`	emparejar ,
8	`i R ] R $`	`i],i$`	emparejar i
9	`R ] R $`	`],i$`	emitir R → ε
10	`] R $`	`],i$`	emparejar]
11	`R $`	`,i$`	emitir R → ,iR
12	`, i R $`	`,i$`	emparejar ,
13	`i R $`	`i$`	emparejar i
14	`R $`	`$`	emitir R → ε
15	`$`	`$`	aceptar

Cuadro 6: Tabla de análisis SLR(1) (sección 4.6.4)

	ACTION					GOTO		
Estado	,	[	]	i	$	L	R	S
0		s3		s4		1		2
1					r1			
2					acc			
3		s3		s4		5		
4	s6		r5		r5		7	
5			s8					
6				s9				
7			r2		r2			
8	s6		r5		r5		10	
9	s6		r5		r5		11	
10			r3		r3			
11			r4		r4			

Cuadro 8: Análisis SLR(1) de `[i,i],i` (sección 4.6.3)

Paso	Pila (símbolo y estado)	Entrada	Acción
1	`0`	`[i,i],i$`	desplazar 3
2	`0 [ 3`	`i,i],i$`	desplazar 4
3	`0 [ 3 i 4`	`,i],i$`	desplazar 6
4	`0 [ 3 i 4 , 6`	`i],i$`	desplazar 9
5	`0 [ 3 i 4 , 6 i 9`	`],i$`	reducir R $\to \varepsilon$
6	`0 [ 3 i 4 , 6 i 9 R 11`	`],i$`	reducir R $\to$,iR
7	`0 [ 3 i 4 R 7`	`],i$`	reducir L $\to$ iR
8	`0 [ 3 L 5`	`],i$`	desplazar 8
9	`0 [ 3 L 5 ] 8`	`,i$`	desplazar 6
10	`0 [ 3 L 5 ] 8 , 6`	`i$`	desplazar 9
11	`0 [ 3 L 5 ] 8 , 6 i 9`	`$`	reducir R $\to \varepsilon$
12	`0 [ 3 L 5 ] 8 , 6 i 9 R 11`	`$`	reducir R $\to$,iR
13	`0 [ 3 L 5 ] 8 R 10`	`$`	reducir L $\to$ [L]R
14	`0 L 1`	`$`	reducir S $\to$ L
15	`0 S 2`	`$`	aceptar

3.7. Sesión de consola

Esta es una sesión completa de `cli/main.py -sets`, con el formato del enunciado. Primero la entrada:

```
3
S -> L
L -> iR [L]R
R -> ,iR e
T
i
[i,i],i
i,
[i

B
[[i]],i
```

```
i,,i

Q
```

y la salida del programa:

```
First(L) = {[, i}
First(R) = {,, e}
First(S) = {[, i}
Follow(L) = {], $}
Follow(R) = {], $}
Follow(S) = {$}
Select a parser (T: for LL(1), B: for SLR(1), Q: quit):
yes
yes
no
no
Select a parser (T: for LL(1), B: for SLR(1), Q: quit):
yes
no
Select a parser (T: for LL(1), B: for SLR(1), Q: quit):
```

4. Ítem 3: definición dirigida por la sintaxis

4.1. La gramática y el SDD

El SDD calcula la suma de verificación de Luhn de una cadena de dígitos. Contando desde el dígito de la derecha (posición $k = 0$), cada dígito en posición impar se duplica, y si el doble pasa de 9 se le resta 9. La cadena es válida cuando la suma total es múltiplo de 10. Esta definición no aparece en el libro ni en el enunciado.

$$\begin{array}{rcl} S & \to & L \\ L & \to & DM \\ M & \to & L \mid \varepsilon \\ D & \to & \texttt{0} \mid \texttt{1} \mid \ldots \mid \texttt{9} \end{array}$$

La gramática cumple las condiciones del paso 1. Sigue las convenciones de la sección 2.2. Tiene cuatro no terminales y 14 producciones. L y M son recursivas entre sí, porque $L \to DM$ y $M \to L$. El programa la clasifica como LL(1) y SLR(1), y analizamos las cadenas con nuestros propios analizadores.

4.2. Tabla de atributos

El SDD tiene atributos sintetizados y uno heredado, `D.k`. El símbolo inicial S no tiene atributos heredados. El atributo principal es `S.valid`.

4.3. Reglas semánticas

El programa guarda estas reglas como datos y las valida con las condiciones de la sección 5.1.1: cada regla define un atributo sintetizado de la cabeza o un atributo heredado de un símbolo del cuerpo, y cada uno de esos atributos queda definido exactamente una vez por producción. El evaluador no conoce este SDD en particular. Para cambiar de SDD basta con cambiar los datos.

Símbolo	Atributo	Clase	Tipo	Significado
S	`sum`	sintetizado	entero	Suma total de los dígitos ya ajustados.
S	`valid`	sintetizado	booleano	Verdadero si `sum` es múltiplo de 10.
L	`cnt`	sintetizado	entero	Cantidad de dígitos de la subcadena que deriva L.
L	`sum`	sintetizado	entero	Suma de los dígitos ajustados de esa subcadena.
M	`cnt`	sintetizado	entero	Cantidad de dígitos de la subcadena que deriva M.
M	`sum`	sintetizado	entero	Suma de los dígitos ajustados de esa subcadena.
D	`k`	**heredado**	entero	Posición del dígito contando desde la derecha (0 para el último).
D	`s`	sintetizado	entero	Valor del dígito: duplicado, y menos 9 si pasa de 9, cuando `k` es impar.
terminal	`lexval`	sintetizado	carácter	El dígito mismo, que da el analizador.

Cuadro 9: Tabla de atributos

Producción	Reglas semánticas
S → L	`S.sum = L.sum` `S.valid = L.sum % 10 == 0`

L → DM	`D.k = M.cnt`
	`L.cnt = M.cnt + 1`
	`L.sum = D.s + M.sum`

M → L	`M.cnt = L.cnt` `M.sum = L.sum`
M → ε	`M.cnt = 0` `M.sum = 0`
D → `d` ($d \in \{0, \ldots, 9\}$)	`D.s = d` si `D.k` es par; `D.s = 2d` si `D.k` es impar y $2d \leq 9$; `D.s = 2d - 9` si `D.k` es impar y $2d > 9$

Cuadro 10: Reglas semánticas del SDD, en el formato de la figura 5.1 del libro

4.4. Paso 3: ejemplo con la cadena `1234`

Con la cadena `1234`, los dígitos se numeran desde la derecha: el `4` tiene $k = 0$, el `3` tiene $k = 1$, el `2` tiene $k = 2$ y el `1` tiene $k = 3$. Se duplican los dígitos con k impar: $3 \cdot 2 = 6$ y $1 \cdot 2 = 2$. Los demás quedan igual. La suma es $4 + 6 + 2 + 2 = 14$, que no es múltiplo de 10, y por eso `S.valid` es falso.

El programa produce este árbol de análisis anotado. Cada nodo muestra su símbolo, un número de nodo y sus atributos.

```
Annotated parse tree:
S#0 [sum=14, valid=False]
  L#1 [cnt=4, sum=14]
    D#2 [k=3, s=2]
      1#3 [lexval=1]
    M#4 [cnt=3, sum=12]
      L#5 [cnt=3, sum=12]
        D#6 [k=2, s=2]
          2#7 [lexval=2]
        M#8 [cnt=2, sum=10]
          L#9 [cnt=2, sum=10]
            D#10 [k=1, s=6]
              3#11 [lexval=3]
            M#12 [cnt=1, sum=4]
```

```
L#13 [cnt=1, sum=4]
  D#14 [k=0, s=4]
    4#15 [lexval=4]
  M#16 [cnt=0, sum=0]
```

El grafo de dependencias tiene una arista por cada atributo que usa una regla, desde el atributo usado hacia el atributo que define. Para `1234` son 32 aristas y 26 atributos por evaluar (los cuatro `lexval` ya se conocen); la figura 1 las dibuja todas. El programa imprime la lista completa con `python cli/sdd_cli.py -graph`.

Un orden de evaluación válido (orden topológico, sección 5.2.2) es el que usa el programa:

1. Los contadores, de abajo hacia arriba: `M#16.cnt`, `L#13.cnt`, `M#12.cnt`, `L#9.cnt`, `M#8.cnt`, `L#5.cnt`, `M#4.cnt`, `L#1.cnt`.
2. La posición y el valor de cada dígito, de izquierda a derecha: `D#2.k`, `D#2.s`, `D#6.k`, `D#6.s`, `D#10.k`, `D#10.s`, `D#14.k`, `D#14.s`.
3. Las sumas, de abajo hacia arriba, hasta el atributo principal: `M#16.sum`, `L#13.sum`, `M#12.sum`, `L#9.sum`, `M#8.sum`, `L#5.sum`, `M#4.sum`, `L#1.sum`, `S#0.sum`, `S#0.valid`.

Resultado: `Main attribute S.valid = False`.

La figura 1 dibuja el árbol anotado de `1234` con su grafo de dependencias encima, como la figura 5.7 del libro. Los nodos del árbol son los símbolos, unidos por líneas grises. Cada recuadro es un vértice del grafo: un atributo con su valor. Las flechas azules van del atributo que se usa al atributo que se define. Se ve el orden de arriba: primero los `cnt` suben desde M_{16}, luego cada `M.cnt` baja a la izquierda hacia el `D.k` de su hermano, y al final las sumas `s` y `sum` suben hasta `S.valid`. Las hojas solo aportan su `lexval`, que ya se conoce.

4.5. Paso 4: clasificación

El SDD no es S-atribuido ni L-atribuido.

- Un SDD S-atribuido solo tiene atributos sintetizados (sección 5.2.3). Este SDD tiene el atributo heredado `D.k`, así que no cumple la definición.
- En un SDD L-atribuido, un atributo heredado de un símbolo del cuerpo solo puede depender de atributos heredados de la cabeza o de atributos de los símbolos que están a su izquierda en la producción (sección 5.2.4). En $L \to DM$ la regla `D.k = M.cnt` hace que D dependa de M, que está a su derecha. Por eso tampoco es L-atribuido.

El evaluador usa el método general del grafo de dependencias, que no pide ningún recorrido fijo del árbol. Por eso evalúa este SDD sin problema.

4.6. Paso 6: pruebas y detección de circularidad

Probamos el evaluador con ocho cadenas. Incluyen la más corta del lenguaje, una cadena fuera del lenguaje y una cadena con recursión profunda.

Para comprobar la detección de ciclos cambiamos temporalmente una regla. En $L \to DM$ la regla `D.k = M.cnt` pasa a ser `D.k = M.cnt + D.s`. Ahora `D.k` depende de `D.s`, que ya dependía de `D.k`. El evaluador detecta el ciclo y se detiene. Esta es la salida con la cadena `18` y la opción `-circular`:

```
The SDD is circular. Cycle found:
  D#2.s -> D#2.k -> D#2.s
```

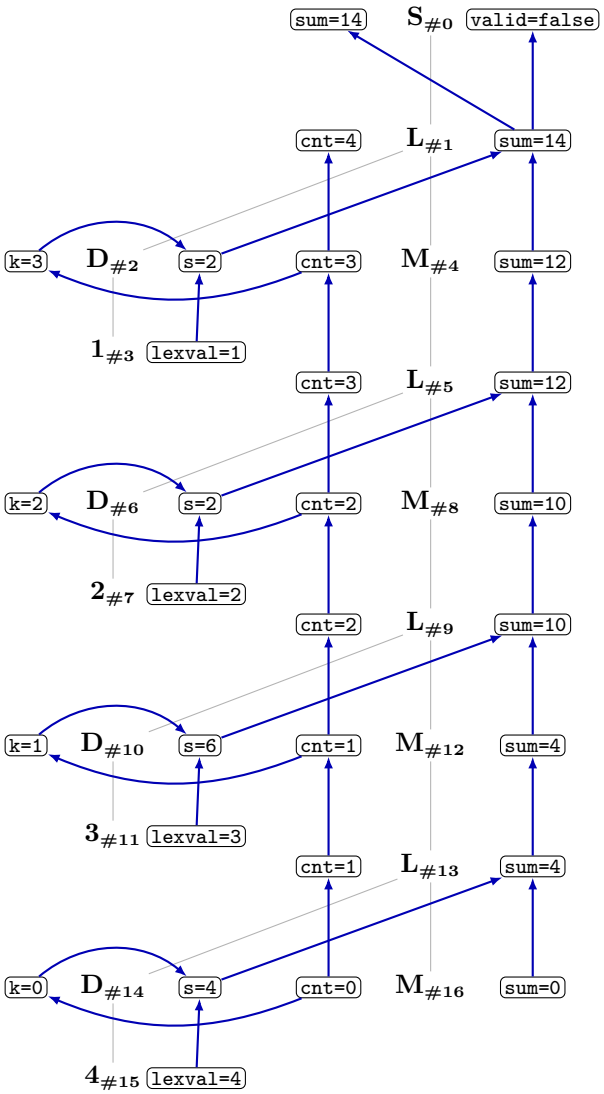


Figura 1: Árbol anotado de `1234` y su grafo de dependencias (secciones 5.1.2 y 5.2.1)

5. Problemas que encontramos y cómo los resolvimos

- **Error 500 con árboles profundos en la web.** Al evaluar 400 dígitos desde la página, el servidor devolvía un error 500, aunque la consola funcionaba. El codificador JSON de Python recorre el árbol con una llamada recursiva por nivel y se pasaba del límite de recursión (1000). Un árbol anotado tiene unos tres niveles por símbolo. Subimos el límite de recursión y el tamaño de pila de los hilos del servidor, y añadimos pruebas con 400 dígitos y con 1990 letras.

- **Problemas de la revisión final.** Al revisar el proyecto terminado contra el enunciado encontramos tres fallos. Los nombres de los estados del DFA en `/convert` salían en orden alfabético (`0-10-2`) en lugar del orden natural (`0-2-10`). El servidor devolvía un error 500 cuando la colección LR(0) tenía más de 400 estados. Y `/convert` y `/simulate` necesitaban comprobar mejor su entrada. Corregimos el orden, el error 500, y añadimos validaciones de entrada y un límite de 1000 estados.

Al programar los algoritmos, estos fueron los puntos más delicados y la forma en que los resolvimos:

- **FIRST de una cadena con símbolos anulables.** $\mathrm{FIRST}(X_1 X_2 \cdots X_n)$ no es solo $\mathrm{FIRST}(X_1)$:

Cadena	Resultado	Comentario
`0`	válida	la cadena más corta del lenguaje
`7`	no válida	un dígito
`1234`	no válida	ejemplo del paso 3
`79927398713`	válida	número de prueba de Luhn
`4539578763621486`	válida	16 dígitos
`12a`	no pertenece al lenguaje	contiene un símbolo que no es dígito
0^{40}	válida	40 ceros
1^{400}	válida	400 unos: recursión profunda (es también el límite de la interfaz web)

Cuadro 11: Pruebas del evaluador con la suma de Luhn (`S.valid`)

si X_1 deriva ε hay que seguir con X_2, y así sucesivamente, y ε solo entra al conjunto si todos los X_i la derivan. Lo implementamos en una función que recorre la cadena y se detiene en el primer símbolo no anulable. Como los conjuntos de los no terminales dependen unos de otros, aplicamos las reglas en rondas hasta que ningún conjunto crece.

- **Regla 3 de FOLLOW.** Es fácil pasar FOLLOW(A) a B solo cuando B está al final de $A \to \alpha B$, pero la regla también aplica a $A \to \alpha B \beta$ cuando β deriva ε. En el ejemplo 2 del enunciado ($S \to AB$, $B \to bBc \mid \varepsilon$) esto es lo que pone \$ en FOLLOW(A). Por eso calculamos FIRST(β) y aplicamos la regla 3 cuando contiene ε.

- **Conflictos SLR(1) que se ocultaban.** Si cada celda de ACTION guarda una sola acción, una acción nueva reemplaza a la anterior y el conflicto desaparece sin aviso. Por eso cada celda guarda la lista de todas sus acciones, y la gramática es SLR(1) solo si ninguna lista tiene más de una. Hacemos lo mismo con la tabla LL(1), aunque la decisión LL(1) usa la condición de la sección 4.4.3.

- **Atributos de las hojas en el orden topológico.** Si las aristas que salen de un `lexval` cuentan como dependencias pendientes en el algoritmo de Kahn, los atributos que las usan nunca quedan listos. Los `lexval` ya se conocen, así que esas aristas no cuentan en el grado de entrada.

- **Símbolos repetidos en una producción.** En una regla como $L \to DM$ basta el nombre del símbolo, pero en una producción con un símbolo repetido el nombre es ambiguo. Las reglas pueden usar `$0` para la cabeza y `$1`, `$2`, ... para el cuerpo, y el programa rechaza un nombre ambiguo.

No pudimos probar el `Dockerfile` ni la integración continua (`.github/workflows`), porque no teníamos un servicio de Docker disponible. Son extras que no pide el enunciado.

6. Verificación y reproducción

El proyecto tiene una suite de pruebas con `pytest`. Incluye una prueba de propiedades que compara los dos analizadores con una enumeración exhaustiva del lenguaje de gramáticas aleatorias. Los pasos para ejecutar las pruebas, los programas de consola y el servidor están en el `README.md`. Los comandos principales son:

```
python -m pytest tests/
python cli/main.py --sets < examples/ex2.txt
python cli/sdd_cli.py < examples/sdd_strings.txt
python cli/sdd_cli.py --circular < examples/sdd_circular.txt
python src/app.py
```

Referencias

[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2.ª ed., Addison-Wesley, 2006.

[2] Enunciado del proyecto final, SI2002 / ST0270 / CM0081, 9 de octubre de 2026.